

8085 Assembly from Absolute Zero

The 9-Phase Core Framework:
Building Crystal Clear Concepts

***Course:** Microprocessor Fundamentals*

***Methodology:** Incremental Concept Building*

A Programmer's Blueprint for Exam Mastery

Contents

1	The Core Phases of the Machine	2
1.1	Phase 1: The Pockets (Data Transfer)	2
1.2	Phase 2: The Calculator (Basic Math)	2
1.3	Phase 3: The Warehouse (Memory)	3
1.4	Phase 4: Making Decisions (Loops)	3
2	Phase 5: The Detective (Logic & Flags)	5
2.1	The Compare Instruction (CMP)	5
2.2	Logical Masking (AND, OR, XOR)	5
3	Phase 6: The Notepad (The Stack)	7
4	Phase 7: The Stopwatch (Time Delays)	8
5	Phase 8: The Heavy Lifter (16-bit Math)	9
6	Phase 9: The Intercom & Fire Alarm (I/O & Interrupts)	10
6.1	The Intercom (I/O Ports)	10
6.2	The Fire Alarm (Interrupts)	10
7	Phase 10: The Mailroom (Hardware Interfacing)	12
7.1	The Concept: Chip Select ($\overline{CS}$)	12
7.2	Solving the Classic Exam Question	12
8	Exam Application: Multiplication	14

Chapter 1

The Core Phases of the Machine

Do not try to memorize code. The 8085 microprocessor is just a machine that follows basic rules. If you understand these phases, you can write any code yourself.

1.1 Phase 1: The Pockets (Data Transfer)

Imagine the CPU is a person wearing a jacket with 7 small pockets. These pockets are called **Registers**, named **A, B, C, D, E, H, and L**. Each pocket can hold one small number (an 8-bit number, like 05H or 12H). Register **A** is called the "Accumulator." It is the most important pocket.

The Rules of Phase 1

MVI (Move Immediate): Means "Put this exact physical number into this pocket."
MOV (Move): Actually means "Copy." It copies a number from one pocket to another. We always read assembly from right-to-left!

Tiny Example 1: Moving Numbers

```
1      MVI A, 05H      ; Put the exact number 5 into pocket A.  
2      MVI B, 02H      ; Put the exact number 2 into pocket B.  
3      MOV C, A        ; Copy the number from A into C. (C is now 5. A  
                        is still 5).
```

1.2 Phase 2: The Calculator (Basic Math)

The CPU has a built-in calculator, but it has a very strict, unbreakable rule: **All math must involve Register A (The Accumulator)**. You cannot add pocket B and pocket C together directly. You must put one number into A, and then add the other one to it. The answer is always spat back into pocket A.

The Rules of Phase 2

ADD B: Means "Take whatever is in A, add B to it, and put the answer back in A."
($A = A + B$)
SUB C: Means "Take whatever is in A, subtract C from it, and put the answer back in A."
($A = A - C$)

Tiny Example 2: Adding 3 and 4

```
1      MVI A, 03H      ; Step 1: Put 3 into the Accumulator.
2      MVI B, 04H      ; Step 2: Put 4 into register B.
3      ADD B           ; Step 3: CPU calculates 3 + 4. Register A now
                        holds 07H.
```

1.3 Phase 3: The Warehouse (Memory)

The pockets (registers) are great, but you only have 7 of them. If you have an array of 100 numbers, you must store them in the **Memory (RAM)**, which is like a massive warehouse outside the CPU.

To get something from the warehouse, the CPU needs an address (like 2000H). Because addresses are 16-bit (big), one pocket isn't big enough. So, we sew pockets **H** and **L** together to make a pointer.

The Rules of Phase 3

LXI H, 2000H: Tells the H and L pockets to team up and point to warehouse address 2000H.

M (The Magic Portal): Whenever you type 'M', the CPU looks at where HL is pointing, runs out to the warehouse, and grabs that exact box.

Tiny Example 3: Read, Change, and Put Back

```
1      LXI H, 2000H    ; Point to warehouse address 2000H.
2      MOV A, M        ; Go to 2000H, grab the number, and bring it
                        into A.
3      INR A           ; "Increment" - secretly add 1 to the
                        Accumulator.
4      MOV M, A        ; Take the new number in A and put it back into
                        2000H.
```

1.4 Phase 4: Making Decisions (Loops)

A program is useless if it just runs top-to-bottom once. We need it to repeat tasks. We do this by setting up a counter in one of our pockets and reducing it until it hits zero.

The Rules of Phase 4

DCR (Decrement): Reduces the number in a pocket by 1.

The Zero Flag: The CPU has a hidden lightbulb. If a DCR instruction causes a pocket to hit exactly 0, the Zero lightbulb turns ON.

JNZ (Jump if Not Zero): Tells the CPU: "If the Zero lightbulb is OFF, jump back up to my bookmark and repeat the code."

Tiny Example 4: A Simple Loop

```
1      MVI C, 03H      ; Put 3 in pocket C. This is our loop
                        counter.
2
3      LOOP:           ; This is a Label (a bookmark). The CPU
                        ignores the word.
4      DCR C           ; Reduce C by 1. (C goes from 3 to 2).
5      JNZ LOOP        ; Is C zero? No. So jump back up to
                        'LOOP:'.
6
7      HLT             ; Once C hits 0, JNZ is ignored, and the
                        program Halts.
```

Chapter 2

Phase 5: The Detective (Logic & Flags)

Sometimes, basic math isn't enough. We need to check if a number is even or odd, or compare two numbers to see which is bigger. To do this, the CPU has a special 8-bit register called the **Flag Register**. Think of it as a dashboard with 5 warning lights.

The two most important lights for you are:

- **Z (Zero Flag)**: Lights up (becomes 1) if the result of an operation is exactly zero.
- **CY (Carry Flag)**: Lights up (becomes 1) if an addition is too big to fit in a pocket, OR if a subtraction requires a borrow (e.g., trying to do 3 - 5).

2.1 The Compare Instruction (CMP)

CMP is the most powerful decision-making tool you have.

How CMP Works

CMP B: The CPU internally calculates (**A - B**). It throws away the mathematical answer, but it updates the warning lights based on what happened!

- If $A == B$: The result is 0. **Zero Flag = 1**.
- If $A < B$: The CPU needs to borrow. **Carry Flag = 1**.
- If $A > B$: Normal math. **Both flags remain 0**.

Tiny Example 5: Which is Bigger?

```
1  MVI A, 05H      ; A gets 5
2  MVI B, 08H      ; B gets 8
3  CMP B           ; CPU does (5 - 8). It needs a borrow! Carry
                     Flag turns ON.
4  JC B_IS_BIGGER  ; "Jump if Carry". Since Carry is ON, it
                     jumps!
```

2.2 Logical Masking (AND, OR, XOR)

Sometimes you only care about a specific bit inside a number. For example, to check if a number is Odd or Even, you only need to look at the very last bit (the Least Significant Bit).

The ANI Instruction (AND Immediate)
ANI 01H: This compares whatever is in pocket A with the number 01H using binary AND logic. It forces all other bits to 0, leaving only the last bit. If the original number was Even, the result becomes exactly 00H, turning ON the Zero Flag!

Chapter 3

Phase 6: The Notepad (The Stack)

You only have 7 pockets. When you write complex programs, you will run out of pockets. The **Stack** is a special, temporary area in the warehouse (RAM) where you can quickly empty your pockets so you can reuse them, and then get your items back later.

The Rules of the Stack (LIFO: Last In, First Out)

PUSH [Pair]: Takes the two items in a register pair (like B and C) and throws them onto the top of the stack pile.

POP [Pair]: Takes the top two items off the stack pile and puts them back into the register pair.

Rule: If you PUSH B and then PUSH D, you MUST POP D before you POP B!

Tiny Example 6: Saving Data

```
1      MVI B, 05H
2      MVI C, 02H
3      PUSH B          ; Throw B and C onto the notepad (Stack)
4
5      ; Now we can use B and C for other math without losing our 5
6      ; and 2!
7      MVI B, 99H
8
9      POP B           ; Get our old data back. B is 05H and C is 02H
10     again!
```

Chapter 4

Phase 7: The Stopwatch (Time Delays)

In the real world, the 8085 runs so fast (millions of instructions per second) that if it turned an LED on and off, human eyes wouldn't even see it. We often need to force the CPU to pause.

We do this by making the CPU do "busy work." We load a massive number into a register pair and tell it to count down to zero.

The Delay Logic

The Loop inside a Loop: An 8-bit pocket can only hold up to 255 (FFH). If counting from 255 to 0 is too fast, we use a 16-bit pocket pair (like BC) which can count from 65,535 (FFFFH) down to 0!

Tiny Example 7: Wasting Time

```
1  DELAY:  LXI B, FFFFH      ; Load BC pair with the maximum 16-bit
           number
2  LOOP:   DCX B             ; Decrement the BC pair by 1
3          MOV A, B          ; Copy B into A
4          ORA C              ; Logically OR A with C.
5          ; (If both B and C are 0, the Zero Flag turns ON!)
6          JNZ LOOP          ; If not zero, jump back and keep
           counting down
7          RET               ; Return to the main program once the
           delay is over
```

Chapter 5

Phase 8: The Heavy Lifter (16-bit Math)

Phase 2 (The Calculator) taught you 8-bit math using just the Accumulator. But what if you need to add two massive 16-bit memory addresses or data values (like 2000H + 3000H)? They won't fit in a single pocket.

Imagine you are playing a video game, and your character's score is **50,000**. An 8-bit pocket can only hold up to **255**. If you try to add **1,000** bonus points, it simply won't fit in the Accumulator (Pocket A). This is why we sew pockets together.

The DAD Instruction (Double Add)

DAD B: This is the ultimate heavy lifter. It treats the **H and L** pockets as a giant 16-bit Accumulator. It takes the 16-bit number in the BC pair, adds it to the HL pair, and leaves the giant answer inside HL.

Tiny Example 8: Adding 16-bit Numbers

The Scenario: Add the 16-bit number 1050H to 0220H.

```
1  LXI H, 1050H      ; Load the first giant number (1050H) into
   the HL pair
2  LXI B, 0220H      ; Load the second giant number (0220H) into
   the BC pair
3
4  DAD B              ; "Double Add BC". The CPU adds BC into HL.
5                      ; Math: 1050H + 0220H.
6                      ; The HL pair now holds the final answer:
                      1270H!
```

Chapter 6

Phase 9: The Intercom & Fire Alarm (I/O & Interrupts)

So far, our CPU is trapped in its own head, only talking to its Memory Warehouse. Exams will ask how it talks to a keyboard, a motor, or a display monitor.

6.1 The Intercom (I/O Ports)

External devices don't live in the memory warehouse. They are connected to "Ports" (addresses from 00H to FFH).

- **IN 80H:** "Listen." Grabs data from Port 80H (e.g., a keyboard) and puts it in the Accumulator.
- **OUT 81H:** "Speak." Takes whatever is in the Accumulator and pushes it out to Port 81H (e.g., an LED display).

6.2 The Fire Alarm (Interrupts)

What if the CPU is busy running a delay loop, but a user presses the "Emergency Stop" button? The CPU has special pins that act as alarms.

The 5 Hardware Interrupts (Highest to Lowest Priority)
1. TRAP: The absolute highest priority. Cannot be ignored (Non-Maskable). Used for power failures or catastrophic emergencies.
2. RST 7.5
3. RST 6.5
4. RST 5.5
5. INTR: The lowest priority. Can be ignored if the CPU is busy.

When an interrupt fires, the CPU drops what it's doing, jumps to a special emergency address to handle the problem (Interrupt Service Routine), and then uses the **RET** instruction to go back to what it was doing.

Tiny Example 9A: The Echo Chamber (I/O)

Imagine we have an 8-switch panel (DIP switches) connected to Port 80H, and 8 LEDs connected to Port 81H. We want to read the switches and immediately light up the corresponding LEDs.

```

1      IN 80H      ; Listen to Port 80H.
2                  ; The switch positions (e.g., 01010101) go into
3                  ; Register A.
4
5      OUT 81H      ; Speak to Port 81H.
6                  ; Send the exact pattern in A out to the LEDs!
7
8      HLT          ; Stop the program.

```

Tiny Example 9B: The Fire Alarm (Interrupts)

Interrupts don't look like normal code because they are triggered by physical hardware wires, not software commands.

Imagine the CPU is stuck in an infinite loop, just wasting time. Suddenly, a user presses a physical "Emergency" button wired to the **TRAP** pin on the actual physical chip.

```

1      ; --- THE MAIN PROGRAM ---
2  MAIN:  JMP MAIN      ; The CPU is stuck here doing nothing
3                  forever.
4
5      ; --- THE EMERGENCY PROTOCOL ---
6      ; The moment the TRAP button is pressed, the CPU automatically
7      ; pauses
8      ; the MAIN loop and jumps to a hardcoded emergency address
9      ; (0024H).
10
11      ORG 0024H      ; This tells the assembler: "Put this next
12                      ; code at 0024H"
13
14  TRAP_ROUTINE:
15      MVI A, FFH      ; Load Accumulator with FFH (11111111 in
16                      ; binary)
17      OUT 99H          ; Send it to Port 99H to turn ON all
18                      ; emergency sirens!
19      RET              ; "Return". The emergency is handled.
20                      ; Go back to the MAIN loop where you were
21                      ; interrupted.

```

Chapter 7

Phase 10: The Mailroom (Hardware Interfacing)

Writing software is only half the battle. The CPU needs to physically connect to memory chips and I/O devices. Since multiple chips share the same Data Bus, we must ensure only ONE chip is awake at a time. We do this using **Address Decoding**.

7.1 The Concept: Chip Select ($\overline{CS}$)

Memory chips have a special power button called Chip Select ($\overline{CS}$). It is active-low, meaning it turns ON when it receives a '0'. We use a decoder chip (like the 74LS138) to act as a security guard. It looks at the upper address lines and sends a '0' only to the correct chip.

The 74LS138 Decoder

This chip takes 3 inputs (usually A_{15}, A_{14}, A_{13}) and has 8 outputs (Y_0 to Y_7).

- If inputs are 0 0 0, output Y_0 goes LOW (0).
- If inputs are 0 0 1, output Y_1 goes LOW (0).
- If inputs are 0 1 0, output Y_2 goes LOW (0).

7.2 Solving the Classic Exam Question

The Problem: Interface a 2K RAM chip at base address 2000H and another 2K RAM at 4000H using partial decoding.

Step 1: Determine the Chip Size Math

A 2K RAM requires 11 address lines because $2^{11} = 2048$. Therefore, lines A_{10} to A_0 go directly into the RAM chip. We use the top lines (A_{15}, A_{14}, A_{13}) to connect to our 74LS138 Decoder.

Step 2: Write the Binary for the Base Addresses

Let's look at the starting address for both chips to see what the decoder needs to look for:

RAM 1 (Base 2000H):

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0

Result: The decoder inputs (A_{15}, A_{14}, A_{13}) are 0 0 1. This triggers output Y_1 . We wire Y_1 to the $\overline{CS}$ of RAM 1.

RAM 2 (Base 4000H):

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Result: The decoder inputs (A_{15}, A_{14}, A_{13}) are 0 1 0. This triggers output Y_2 . We wire Y_2 to the $\overline{CS}$ of RAM 2.

Step 3: The I/O Control Signals

Memory chips and I/O devices must be kept separate. The 8085 has a pin called $IO/\overline{M}$.

- When $IO/\overline{M} = 0$, the CPU is talking to Memory. We connect this to the Enable pin of our Memory Decoder.
- When $IO/\overline{M} = 1$, the CPU is talking to I/O Ports (like the 17H and 07H in the question). We use this to enable a separate I/O Decoder.

Chapter 8

Exam Application: Multiplication

The 8085 microprocessor *does not have a multiply button*. You cannot just say `MUL B`. So, how do we solve this? We have to remember elementary school: multiplication is just repeated addition.

If the numbers are 5×4 , we just need to add the number 5 to itself, 4 times ($0 + 5 + 5 + 5 + 5 = 20$).

The Problem: Write an 8085 assembly language program to multiply two 8-bit numbers stored at memory locations 2000H and 2001H. Store the result at 2002H.

Step-by-Step Code Construction

Step 1: Setup the Warehouse and Pockets

```
1      LXI H, 2000H      ; Phase 3: Point to the first number in the
                        ; warehouse.
2      MOV D, M          ; Phase 1: Copy the first number into pocket
                        ; D (This is our "5").
3
4      INX H             ; Move pointer to the next address (2001H).
5      MOV C, M          ; Phase 1: Copy the second number into
                        ; pocket C (This is our counter, "4").
6
7      MVI A, 00H        ; Clear the Accumulator to 0 so we can start
                        ; adding!
```

Step 2: The Loop (Repeated Addition) & Step 3: Store Result

```
1 MULTIPLY_LOOP:        ; This is our bookmark.
2      ADD D             ; Phase 2: Add the number in D to the
                        ; Accumulator. (A = A + D)
3
4      DCR C             ; Phase 4: We did one addition, so decrease
                        ; our counter (C).
5      JNZ MULTIPLY_LOOP ; Phase 4: If the counter is not zero,
                        ; jump back and add again!
6
7      ; If the CPU reaches here, the loop is done. The final
                        ; answer is sitting in A.
8      INX H            ; Move pointer to the destination address
                        ; (2002H).
9      MOV M, A          ; Phase 3: Put the grand total back into the
                        ; warehouse.
10     HLT               ; Stop the program.
```

Summary of the Full Exam Code

When you write it on your exam paper, it looks clean and professional like this:

```
1      LXI H, 2000H
2      MOV D, M
3      INX H
4      MOV C, M
5      MVI A, 00H
6 MULTIPLY_LOOP:
7      ADD D
8      DCR C
9      JNZ MULTIPLY_LOOP
10     INX H
11     MOV M, A
12     HLT
```